IOAI 2025 Individual Contest Day 1

Team Leader's Translation Version

Team Leaders must complete the form below, even if they don’ t need translated documents.

Country Or Region	Singapore
Number of Team Members (sample papers will be duplicated according to the number)	8
How many pages in total (cover pages included)	25

IOAI2025 On Site Stage: Radar

Note: Please "join" the competition first. Then, you can mount the dataset to the GPU. Otherwise, the notebook may encounter an error because it cannot access the dataset until you have joined the competition.

1. Problem Description

Radar is a key technology in wireless communication, with widespread applications such as self-driving cars. It typically involves an antenna that transmits specific signals and receives their reflections from objects in the environment. By processing these signals, the system determines the angular direction, distance, and velocity of target objects.

In real-world applications, radar signal processing is challenging due to noise and reflections from non-target objects in the surroundings. For example, when attempting to detect pedestrians, the radar may also receive reflections from trees or other background objects, which can degrade accuracy. Your task is to use AI to analyze the signals received by the radar and identify the presence of a human at each position.

In this task, we provide an **indoor radar experiment dataset**, and your objective is to develop a model that performs **radar semantic segmentation**.

2. Dataset

To measure objects surrounding a radar, the following key parameters are used:

- **Range**: The straight-line distance between the radar and an object.
- **Azimuth**: The horizontal angle (left to right) between the radar and the object.
- **Elevation**: The vertical angle (up or down) of the object relative to the radar.
- **Velocity**: The speed at which the object is moving toward or away from the radar.

The radar data is processed into multiple **heatmaps**, each encoding the **received signal strength** at various positions and directions.

- **Static heatmaps** emphasize reflections from **stationary** objects.
- **Dynamic heatmaps** highlight changes caused by **moving** objects.

When no object is present at a specific location, the signal consists mostly of background noise and appears weak. In contrast, reflections from an object increase signal intensity, enabling detection of the object.

For example, the **static range-azimuth heatmap** represents signal strength across different distances (**range**) and horizontal angles (**azimuth**), mainly reflected by stationary objects.

Each sample in the dataset is stored in a .mat.pt file as a tensor of shape $7 \times 50 \times 181$, where:

- 7 is the number of maps (6 heatmaps + 1 semantic label map),
- 50 represents range bins (distance),
- 181 represents angular or velocity bins, covering angles from -90° to +90° in either the horizontal or vertical plane. You can assume that the velocity bins are also remapped from -90° to +90° for visualization consistency.
- each heatmap intensity value is normalized to [0, 1], representing received signal strength.

The 6 heatmaps are structured as follows:

- **Index 0**: Static range-azimuth heatmap
- **Index 1**: Dynamic range-azimuth heatmap
- **Index 2**: Static range-elevation heatmap
- **Index 3**: Dynamic range-elevation heatmap
- **Index 4**: Static range-velocity heatmap
- **Index 5**: Dynamic range-velocity heatmap

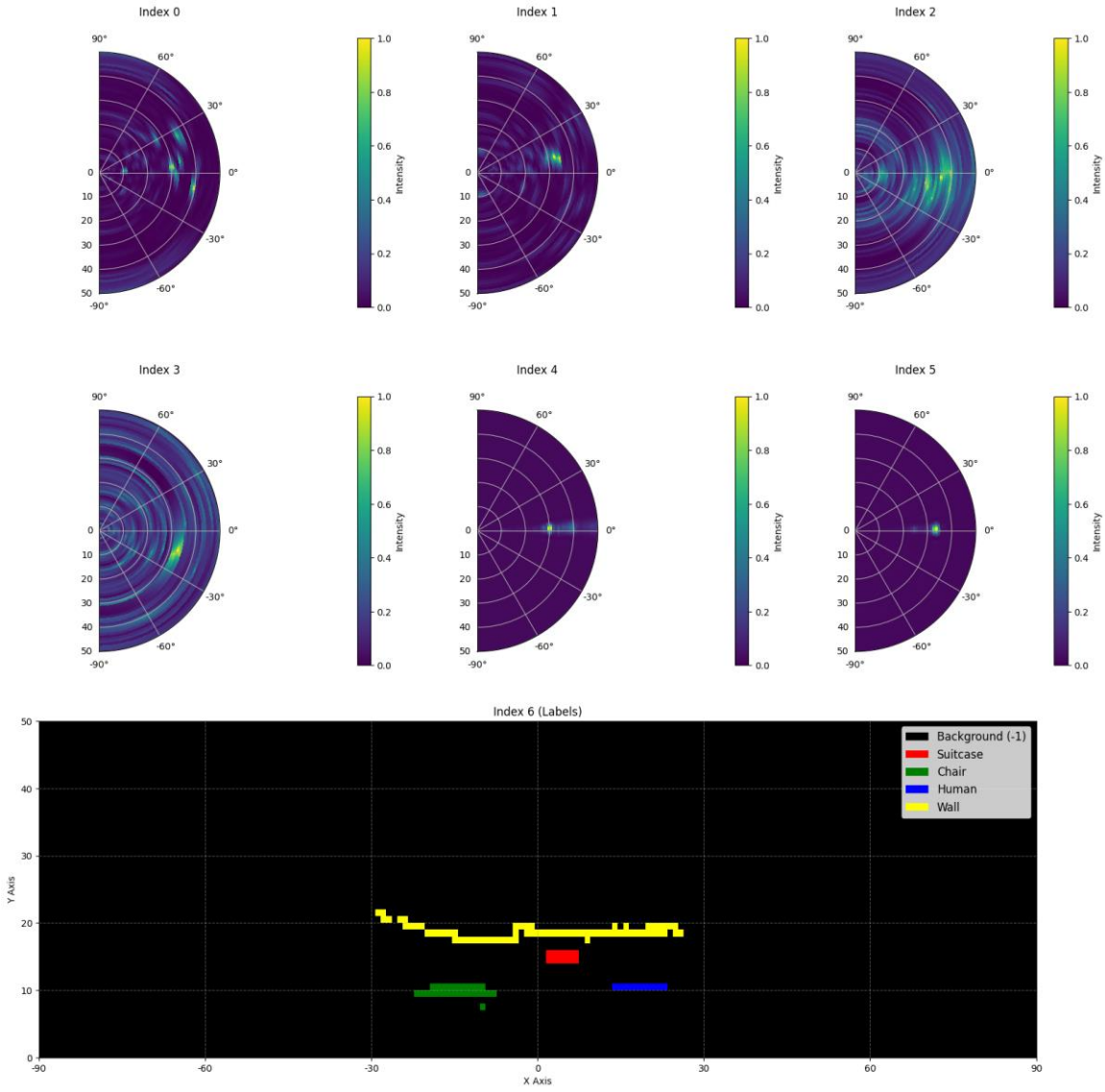
All values in heatmaps are **normalized**, so no unit conversion is required.

The **map at Index 6** is the semantic label map, stored in range-azimuth format.

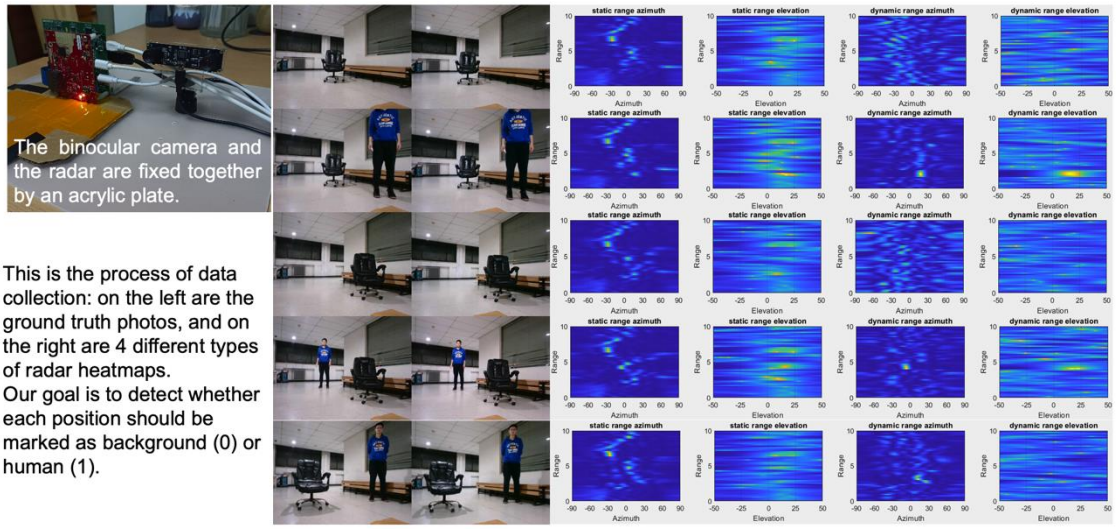
- **-1**: Background (no target)
- **0**: Suitcase
- **1**: Chair
- **2**: Human
- **3**: Wall

This is the visualization of 1.mat.pt in training_set:

Sign your name: Pan Xingang



Here is part of a sample from the dataset:



Ground Truth

Data scale: 1800 samples in the training set, 500 samples in the validation set, and 500 samples in the test set.

3. Task

Your task is to develop a model that takes the **first six heatmaps** (indices 0 to 5) as input, and predicts the **semantic label map** (index 6) as the output. The goal is to accurately identify what the target is (-1 to 3) at each location in the radar's field of view.

1. **Input**: A tensor of shape $6 \times 50 \times 181$, representing six radar heatmaps.
2. **Output**: A tensor of shape 50×181, representing the target semantic label map.

4. Submission

Please submit a file named submission.ipynb. The output is a zip file named "submission.zip", which contains two tables submission_val.csv and submission_test.csv corresponding to the prediction results of the validation set and the test set respectively. (csv means Comma Separated value. In the .csv file, we use comma to separate cells of a table. For example, "1, 2, 3" means three cells | 1 | 2 | 3 |.)

Note: The output table should have a header, the data in the table is not the actual solved data, it is only used as an example of the submission format.

Relationship between the 50×181 image and the 9049 pixels in the table: the 50×181 is flattened to 9049 by stacking the 50 rows to one row (i.e., row-first scan).

filename	pixel_0	pixel_1	...	pixel_9049
1.mat.pt	-1	-1	...	-1
...	...	...	...	...

5. Score

The score is based on the **accuracy of label recognition**. Correctly identifying target points is weighted more heavily than correctly identifying background points.

Scoring Criteria:

- Each correctly identified **background pixel** earns **1 point**.
- Each correctly identified **non-background pixel** earns **50 points**.
- The final score is normalized to a **0-1 point** by comparing it to the maximum possible score.

Formula:

$$Score = \frac{\left|C_{0,correct}\right| \times 1 + \left|C_{1,correct}\right| \times bonus}{|C_0| \times 1 + |C_1| \times bonus}$$

where:

$$\begin{aligned} I &= \{1,2,\dots,50 \times 181\} \\ C_0 &= \{i \in I \mid y_i = -1\} \\ C_1 &= \{i \in I \mid y_i \neq -1\} \\ C_{0,correct} &= \{i \in C_0 \mid p_i = y_i\} \\ C_{1,correct} &= \{i \in C_1 \mid p_i = y_i\} \end{aligned}$$

Example

For a 3x3 heatmap, assume the Ground Truth is

$$\begin{bmatrix} -1 & -1 & -1 \\ 1 & 2 & 3 \\ -1 & -1 & -1 \end{bmatrix}$$

The result you identified is

$$\begin{bmatrix} -1 & 1 & -1 \\ -1 & 2 & -1 \\ -1 & 3 & -1 \end{bmatrix}$$

Then there are four correctly identified -1 and one correctly identified 2. Your score is 4 + 50 = 54 points. The maximum possible score is $6 + 50 \times 3 = 156$, that is, the score for six background pixels and three non-background pixels. Your normalized score is 54 / 156 = 0.346.

$$Score = \frac{4 \times 1 + 1 \times 50}{6 \times 1 + 3 \times 50} = 0.346$$

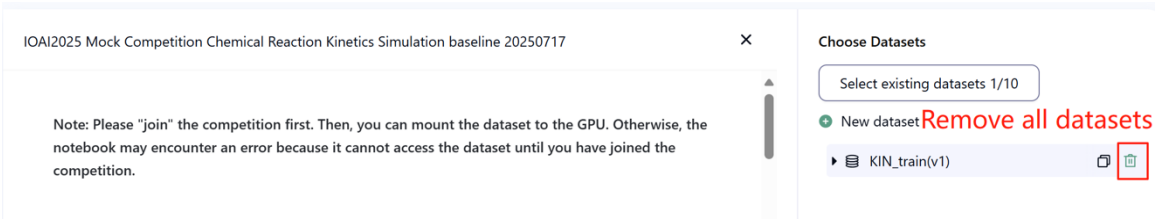
6. Baseline and Training Set

- The baseline is in **baseline.ipynb**.

- The dataset is in **training set**.

7. Requirements

- Maximum submission limit: **15 times**. Only successful submissions (i.e., those receive a score on Leaderboard A) will be counted toward the submission limit.
- Testing environment restrictions: The test machine will run your Notebook within **15 minutes**. If the execution time exceeds **15 minutes**, the system will forcibly terminate and return a feedback of “Timeout” or “Failed”.
- Data and model submission: In this task, contestants can only submit a Notebook and **cannot** submit any mounted datasets or .pth files generated by themselves. Please check the upper right corner to remove the mounted dataset first.

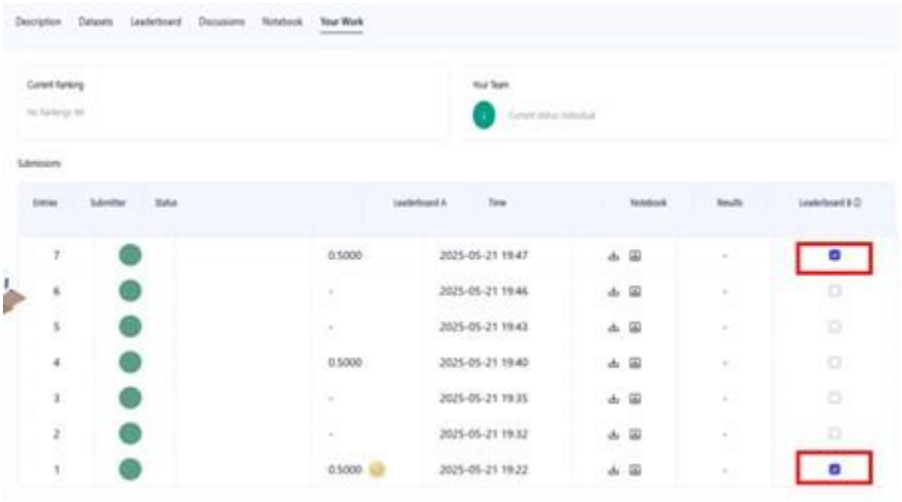


- Network: For the on-site stage, the test machine cannot connect to the internet. In other words, downloading commands such as 'pip' and 'conda' or trying to call APIs will not work.
- Pretrained Model: Any pre-trained model can be used in this task when it can be imported properly without network connection and downloading. This message means the Bohrium cannot guarantee to exclude all the pretrained model in the Python image when install the packages, when you find some useful ones, you can use them.

8. Precautions

- Which score is effective: Contestants can select up to 2 submission results for scoring (√ - selected, □ - not selected). The score before unification for this task will be determined by the higher score on the Leaderboard B among the two selected submissions. Other cases of score calculation: please refer to **Appendix Platform Mechanisms and Restrictions for Individual Contest & GAITE**.

Sign your name: Pan Xingang



- How to deal with ambiguity: Once there is a conflict between the task description and the training set, the validation set and test set , the dataset will be respected first, and the dataset will not be changed during the competition.
- Contestants can only access Leaderboard A during the contest and cannot access Leaderboard B. The final score will be calculated only based on the score in Leaderboard B.
- The highest score by the Scientific Committee for this task is 0.90 in Leaderboard B, this score is used for score unification.
- The baseline score by the Scientific Committee for this task is 0.67 in Leaderboard B, this score is used for score unification.

IOAI2025 On Site Stage: Chicken Counting Problem

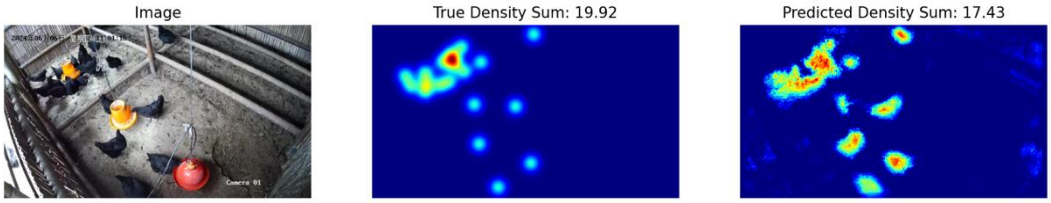
Note: Please "join" the competition first. Then, you can mount the dataset to the GPU. Otherwise, the notebook may encounter an error because it cannot access the dataset until you have joined the competition.

1. Problem Description

As the leader of an AI research team collaborating with Silkie chicken farmers, you are tasked with solving a critical challenge in traditional free-range farming. Accurate counting of livestock is crucial for both farmers and insurance companies, as factors like disease outbreaks and predator invasions can significantly impact the survival rate of these chickens in a short time. While insurance coverage helps mitigate farming risks, the claims process requires precise counting of livestock losses. Your farmers have approached your team for help in developing more accurate, automated counting systems. The challenge before your research team is to develop an optimized Silkie chicken counting model using density estimation techniques that can provide reliable counts to support both farm management and insurance processes.

Your team has access to a pretrained feature extractor for Silkie chicken images, but you'll need to design and train the density estimation decoder to create a complete counting solution. Your task is to build upon this foundation by developing an effective decoder architecture and training strategy to achieve accurate chicken counts that farmers and insurance companies can rely on.

The below figure shows an image in the dataset, as well as the corresponding true density distribution and a predicted density distribution generated by the baseline model. The total density (sum of densities across all areas) is labeled.



2. Dataset

The structure of the provided Silkie chicken image dataset is as follows:

```
datasets/
├── train/
│   └── A dataset with features:
│       ├── `image`: `PIL.Image` with RGB channels (3 × 720 × 1280)
│       └── `density`: a 2D array of shape 180 × 320
└── base.pth (Pretrained Model)
```

```
os.environ.get("DATA_PATH")/
├── test_a/
│   └── A dataset with features:
│       └── `image`: `PIL.Image` with RGB channels (3 × 720 × 1280)
└── test_b/
    └── A dataset with features:
        └── `image`: `PIL.Image` with RGB channels (3 × 720 × 1280)
```

(1) Training set location: datasets. Files in this folder are used for model fine-tuning. It contains a train folder, which stores a dataset with 100 images and their corresponding density maps.

(2) Validation set (test_a) and Test set (test_b): These will be used to evaluate scores on Leaderboard A and Leaderboard B, respectively. They will be inaccessible to contestants. Only the score achieved on test set B will be used for final scoring.

(3) Datasets size:

- Training set: 100 images.
- Validation set: 100 images.
- Test set: 100 images.

(4) Validation set (test_a) and Test set (test_b) are not visible.

(5) Due to limits of computing resources, training density maps are reshaped to $1 \times 180 \times 320$. **NOTE** the output density map can be viewed as a 2D real number matrix with shape 180×320, the sum of all matrix values is the count of chickens.

3. Task

Your task is to train your own model using the training data to predict density maps, thereby serving the purpose of chicken counting.

You may extend and optimize the given pretrained model to improve its count prediction accuracy. The pretrained model base.pth only contains the weights of the first four layers of the model (the feature extraction model), and the function load_pretrained_weights_partial in the baseline code can be used to load these partial weights into your model. You may construct a density decoder Density Decoder and combine it with the pretrained feature extraction module to form a complete data prediction model.

```
class DensityDecoder(nn.Module):
    def __init__(self):
        ################################################
        # Your code here
        ################################################

    def forward(self, x):
```

```
        ##################################################
        # Your code here
        ##################################################
        return x
```

You can also build your own model without the pretrained model we provided.

This task is the continuation of Satellite Weather Forecasting. A kind remind is the UNET is easily to full GPU memory without any feature engineering. Then, the GPU memory error message will be reported.

Please follow these rules to achieve a score normally:

(1) Your model must output the predicted density map.

(2) Due to limits of computing resources, your output density map should be reshaped to 180×320. This is also the shape of target density maps provided in the train dataset.

4. Submission

Please submit a **submission.ipynb** that includes the following components:

（1）**Training Code**

- Include the full training pipeline.

（2）**Evaluation Code**

- Evaluate your model on the validation set and test set. Refer to **baseline.ipynb** on how to access these datasets with a secret DATA_PATH.
- The output should be saved as **submission.npz**. This must be a valid npz file containing two arrays pred_a and pred_b, each with shape $100 \times 1 \times 180 \times 320$ (The evaluation script will also accept predictions in the shape of $100 \times 180 \times 320$, if you decide to squeeze the channel dimension).

  Any result that does not meet the specified size will be considered invalid, resulting in an assessment score of zero.
- Each element of the density map should be **no less than zero**, otherwise will result in an assessment score of zero.

5. Score

You will be scored based on the mean relative error of your model. Relative error is defined by:

$$\text{Relative Error}=\frac{|y_i-\hat{y}_i|}{|y_i|}$$

where y_i is the true total density for the i-th sample, and $\hat{y}_i$ is the predicted total density for the i-th sample.

Your final score before normalization will be calculated based on your mean relative error, as follows:

$$\text{Score}=\exp\left(-\frac{1}{n}\sum_{i=1}^{n}\frac{|y_i-\hat{y}_i|}{y_i}\right)$$

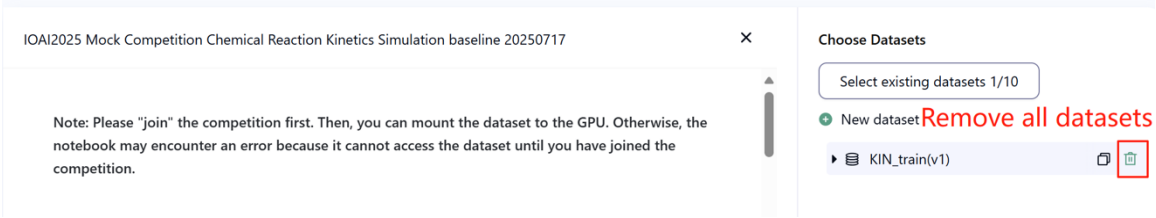
6. Baseline & Training Set

The baseline is in **baseline.ipynb**.

The training set is in **training set**.

7. Requirements

- Maximum submission limit: **15 times**. Only successful submissions (i.e., those receive a score on Leaderboard A) will be counted toward the submission limit.
- Testing environment restrictions: The test machine will run your Notebook within **15 minutes**. If the execution time exceeds **15 minutes**, the system will forcibly terminate and return a feedback of "Timeout" or "Failed".
- Data and model submission: In this task, contestants can only submit a Notebook and **cannot** submit any mounted datasets or .pth files generated by themselves. Please check the upright corner to remove the mounted dataset first.

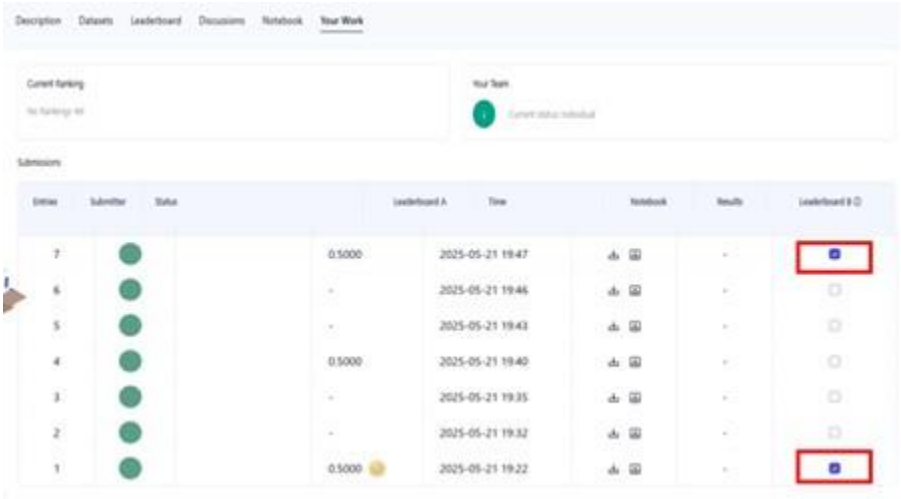


- Network: For the on-site stage, the test machine cannot connect to the internet. In other words, downloading commands such as 'pip' and 'conda' or trying to call APIs will not work.
- Pretrained Model: You may use the provided pretrained weights at **base.pth**, but it's not a must.

  Any other pretrained model can be used in this task when it can be imported properly without network connection and downloading. This message means the Bohrium cannot guarantee to exclude all the pretrained model in the Python image when install the packages, when you find some useful ones, you can use them.

8. Precautions

- Which score is effective: Contestants can select up to 2 submission results for scoring (√ - selected, □ - not selected). The score before unification for this task will be determined by the higher score on the Leaderboard B among the two selected submissions. Other cases of score calculation: please refer to **Appendix Platform Mechanisms and Restrictions for Individual Contest & GAITE**.



- How to deal with ambiguity: Once there is a conflict between the task description and the training set, the validation set and test set , the datasets will be respected first, and all the datasets will not be changed during the competition.

- Contestants can only access Leaderboard A during the contest and cannot access Leaderboard B. The final score will be calculated only based on the score in Leaderboard B.

- The highest score by the Scientific Committee for this task is 0.89, this score is used for score unification.

- The baseline score by the Scientific Committee for this task is 0.71, this score is used for score unification.

IOAI2025 On Site Stage: Concepts

Note: Please "join" the competition first. Then, you can mount the dataset to the GPU. Otherwise, the notebook may encounter an error because it cannot access the dataset until you have joined the competition.

1. Problem Description

Concepts is a word-guessing game where players communicate ideas through visual icons. There are two roles: the **Clue-Giver** and the **Guesser**. A shared set of visual icons, each with a known description, is available to both players. Here are some sample icons:

The Clue-Giver first selects a **secret**, which is a word or phrase, and then provides a **hint** about it by pointing to an **ordered sequence** of icons from the shared set — speaking or writing is not allowed.

The order of the icons in the hint is meaningful:

- The **first icon** typically represents the core idea of the secret.
- The **subsequent icons** provide supporting context that helps clarify or elaborate on the main concept.

Example 1

The following hint might be interpreted as *a place where a job that fights fire takes place* — in other words, a **fire station**:

If the icon order is reversed, it could instead suggest *a job that fights fire in a house* — pointing to a **firefighter**:

Example 2

Icons can take on different meanings depending on their context. For example, the heart icon can appear in the following hint, which suggests *a tool used by doctors to listen to the heart* — a **stethoscope**:

The same heart icon might instead appear in a hint that implies *a fictional character that is both dead and alive* — pointing to a **zombie**:

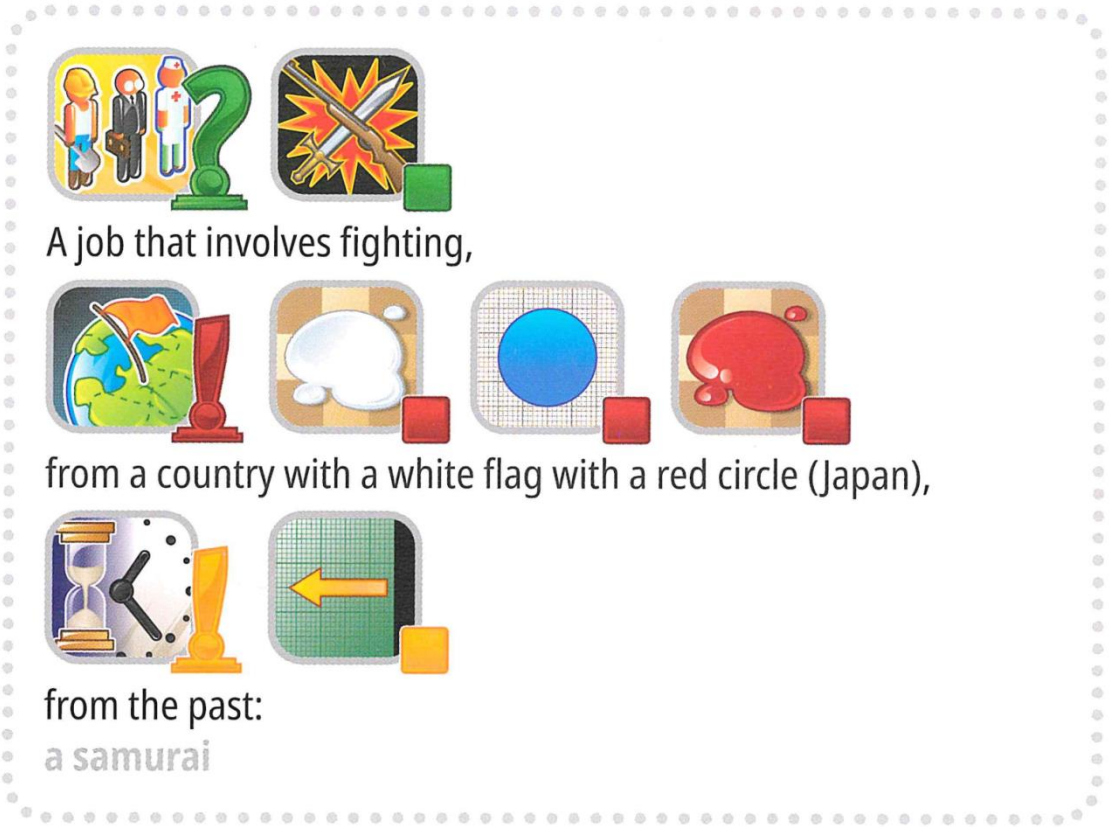
At Home-Stage, contestants had developed an AI program that predicts outcomes based on a sequence of hints. It's fun. However, your friend has now challenged you:
"Guessing is easy, but can you make an AI system that can give a good clue as well?"

In fact, they further challenged you to see if you can make an AI system that can provide a good clue so that another AI system can guess your keyword!

To make the challenge more interesting, we now play the typical Concept game. To recall, our previous Concept game was simplified so that a clue could only consist of a single sequence of markers.

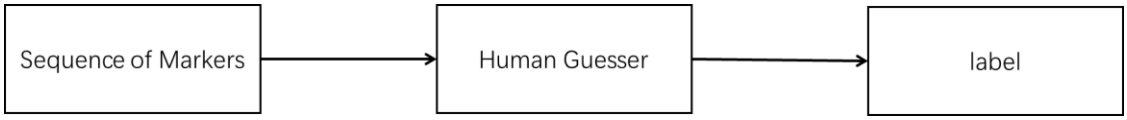
Now, you may provide up to **4 sequences of markers**!

With this, we can express complex ideas better. Considering the following that utilizes 3 sequences of markers to explain a **samurai**:



But wait, there's more. To make the challenge even more interesting, the game will now include some keywords that are not typically present in a Concept game, such as "International Olympiad."

Are you up to the challenge?



New AI-Powered Format:
We've replaced the human guesser with an **AI Guesser**. To streamline game play:

3. The target word (*label*) will always be selected from a predefined set (options).
4. Hints must be an **ordered sequence of markers** chosen exclusively from a fixed set of **118 candidate markers**.

Terminologies

To ensure clarity and consistency, we are standardizing key terms across all materials.

- **label**: The target answer (or "secret") to be identified.
- **options**: The predefined candidate set from which all valid *labels* are selected.
- **marker**: An icon representing a concept, accompanied by its text description.
- **hints**: An **ordered sequence** of *markers* provided to the AI guesser to help identify the *label*.

Your task is to provide **hint** for each *label* to help the **AI guesser** identify it. For example, when the target label is "microphone", your program may generate four hint sequences like:

1. **Hint 1**:
   ["Object-Box", "Electronic-Computing", "Mouth-Taste", "Ear-Sound-Hearing", "Tool-Construction"]
2. **Hint 2**:
   ["Music-Song", "Television-Program-Show", "Work-Occupation", "Use - Action-Do - Verbe-Button"]
3. **Hint 3**:
   ["Black", "Metal", "Plastic-Rubber", "Cylinder", "Circle-Ring"]
4. **Hint 4**:
   ["Arm-Hand-Finger", "Happy-Positive", "Expression - Quote-Talking-Words", "Life-Heart-Love"]

Note: The above example is for illustrative purposes only. The program should generate lists of IDs but not strings; please refer to both `3. Task` and the baseline.

2. Dataset

The structure of the provided dataset is as follows:

```
datasets/
├── train/
│   └── A huggingface dataset
└── hint_descriptions/
    └── A huggingface dataset


os.environ.get("DATA_PATH")/
└── test/
    ├── test_a/
    │   └── A huggingface dataset
    └── test_b/
        └── A huggingface dataset
```

1. Training set: files in this folder are used for model training. It contains:
    - train/: A huggingface dataset with a single split 'train', with 30 examples, each containing:
        - label: string - the target keyword/answer
        - options: sequence of strings - list of 100 possible choices
    - hint_descriptions/: A huggingface dataset with a single split 'train', with 118 markers and their descriptions:
        - ID: int64 - unique identifier for each marker
        - Description: string - textual description of what the marker represents
        - image: Image - visual representation of the marker
2. Test sets: Located at os.environ.get("DATA_PATH")/test/. These will be inaccessible during development and will only be available in the evaluation environment:
    - test_a/: A huggingface dataset with a single split 'test' containing 150 examples for leaderboard A evaluation
    - test_b/: A huggingface dataset with a single split 'test' containing 150 examples for final scoring
      Both test datasets contain the same structure as the training set (label and options fields).

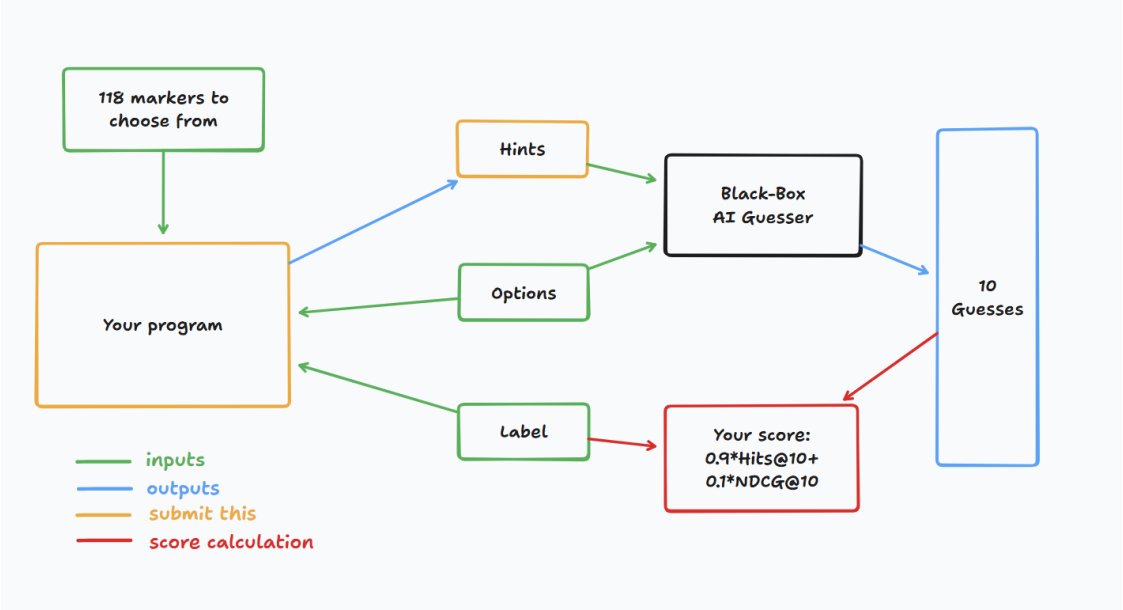
3. Task

Your task is to provide **hint** for each *label* to help the **AI guesser** identify it. You are required to develop a program that takes two inputs:

1. A string label (your secret *label*)
2. The options list (100 candidate choices for the guesser)

The label is guaranteed to be one of the options.

Additionally, the program will utilize the predefined candidate markers.



This program should return a list of lists of integers for each label, representing the hints(i.e. the sequences of markers). Specifically:

- The returned list must contain **no more than 4** sequences.
- Each sequence can contain **up to 8 integers**.
- Each integer represents the ID of a marker in the clue.

Your hints will then be given to a black-box AI guesser. You will score a point if the black-box AI guesser can correctly guess your secret keyword based on your hints.

4. Submission

Submit a notebook that generates submission.zip, which includes clues_a.jsonl and clues_b.jsonl, the clues for testset a and b, respectively, in json format. Refer to the baseline notebook for how to generate these files and the specific structure of the jsonl files. Please make sure to follow the naming and structuring conventions.

Contestants may submit model files. **If submitting model files, contestants must create a corresponding dataset on the Bohrium platform.** Only **one** dataset **can** be submitted for this task, and its size **must not** exceed 2GB. Preloaded datasets and models will be automatically mounted on the test machine, **eliminating the need for manual mounting during submission.**

Additionally, contestants are permitted to use larger external models to assist in developing their submission.

5. Score

Your clue is evaluated using two metrics:

Hits@10

= 1 if the secret word is in the top 10 guesses from the AI, else 0.

NDCG@10 (Normalized Discounted Cumulative Gain)

Rewards the correct guess more if it appears higher in the list.

If the secret word is at rank *i* (1-based):

$$\text{NDCG@10} = \frac{1}{\log_2(i+1)} \qquad (1)$$

Examples:

- Rank 1 → 1.00
- Rank 2 → ~0.63
- Rank 4 → ~0.43
- Rank 10 → ~0.29

Final Score

Your final score will be a combination of both, specifically, it will be 0.9 Hits@10 + 0.1 NDCG@10.

The scoring means that you'll get a significant point as long as the guesser can guess the secret keyword correctly, but more point is given if the guesser can predict the secret keyword earlier.

6. Baseline & Available Tools

- The baseline is in baseline.ipynb
- The training set and pretrained models are in training set

AI-Guesser API

You can assess the AI-guesser for you to play around with. See the following code on how to access the guesser. It is recommended to implement exponential retry logic as network failures might occur. For more comprehensive code, refer to the baseline.ipynb.

```
guesser_response = httpx.post(f"{API_URL}/guess", json={
    "clues": clues,
    "options": options
    }, headers={
        "Authorization": f"Bearer {SCORER_API_KEY}"
    }, timeout=60).json()
```

Important: The AI-Guesser API will not be available on the inference machines. In other words, do **NOT** call the api in your submission notebooks. This is only for you to validate/train your

model locally. You may attach your model weights, training data, etc. via a Bohrium dataset. Refer to the Requirements section.

Environment

We installed vllm, sglang and unsloth in environment for LLM inference and fine tuning. Additionally, we provide access to the following huggingface models via a training set in Bohrium platform:

Embedding Models

sentence-transformers/all-MiniLM-L6-v2
sentence-transformers/all-MiniLM-L12-v2
sentence-transformers/all-mpnet-base-v2
sentence-transformers/paraphrase-mpnet-base-v2
sentence-transformers/paraphrase-MiniLM-L6-v2
intfloat/e5-small
intfloat/e5-base
intfloat/e5-large
intfloat/e5-small-v2
intfloat/e5-base-v2
intfloat/e5-large-v2
intfloat/multilingual-e5-small
intfloat/multilingual-e5-base
Alibaba-NLP/gte-modernbert-base
Snowflake/snowflake-arctic-embed-xs
Snowflake/snowflake-arctic-embed-s
Snowflake/snowflake-arctic-embed-m
Snowflake/snowflake-arctic-embed-m-long
Snowflake/snowflake-arctic-embed-l
BAAI/bge-large-en
BAAI/bge-base-en
BAAI/bge-small-en
BAAI/bge-large-en-v1.5
BAAI/bge-base-en-v1.5
BAAI/bge-small-en-v1.5
WhereIsAI/UAE-Large-V1
mixedbread-ai/mxbai-embed-large-v1

Small LLMs

Qwen/Qwen3-0.6B

Qwen/Qwen2.5-0.5B

Qwen/Qwen2.5-0.5B-Instruct

unsloth/Qwen3-0.6B

facebook/opt-350m

facebook/opt-125m

You may refer to huggingface models tutorial for how to load successfully load these local models.

LLM Proxy Access

We provide a LLM proxy. Reference llm proxy tutorial for how to access the proxy. Note that you will have a $10 credit limit. As a reminder, you are not allowed to use private APIs, login to any external services (including huggingface), or use your own tokens for any service.
Important: The LLM proxy will not be accessible on the test machines. If your submitted notebook tries to call the api, it will not run.

7. Requirements

Overall Requirements

- *Maximum submission limit*: **15 times**. Only successful submissions (i.e., those receive a score on Leaderboard A) will be counted toward the submission limit.
- *Testing environment restrictions*: The test machine will run your Notebook within **10 minutes**. If the execution time exceeds **10 minutes**, the system will forcibly terminate and return a feedback of “Timeout” or “Failed”.
- *Data and model submission*: In this task, participants should submit a Notebook and have the option to submit 1 attached dataset generated by themselves. The dataset must not exceed 2GB in total size. You are warned that attaching a very large dataset might prolong the testing process, as the dataset needs to be mounted to the inference machine. While this process does not count towards the execution time limit, it will take longer for you to be able to see and select your submissions.
- *Network*: For the on-site stage, the test machine cannot connect to the internet. In other words, downloading commands such as 'pip' and 'conda' or trying to call APIs will not work. Specific to this problem, the testing machines cannot access the guessor API nor the LLM proxy.

API Access Limitations

Your token will grant you 12,500 POST requests to /guess endpoint of the AI-guesser API.

- If successful, this will return a dictionary in the form {'guesses': ['firefighter', 'fire inspector', 'fire marshal', 'fire warden', 'fire safety officer', 'smokejumper', 'pyrotechnician', 'firewatcher', 'chef', 'cook'], 'message': 'Generated 10 guesses'}.

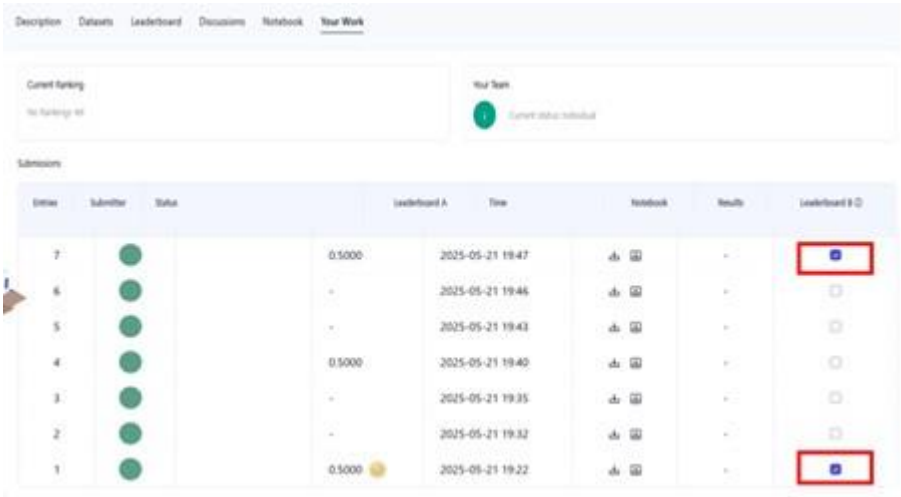
- If unsuccessful, this will either raise an HTTPStatusError and return {'detail': 'error message'}, or return {'guesses': [], 'message': 'unable to generate guesses'}.
- Usually, the former is because you have exceeded the 12,500 limit of your token, or you tried to make more than 1000 requests within 1 minute, and the latter is because your clues exceeded 4 sequences or exceeded 8 markers in 1 sequence.
- Refer to the error message for details.
- There might be 1-2 failures in 1000 requests, 'retry' mechanism is provided in baseline.ipynb.

Warning

The scientific committee will closely monitor the submission notebooks for this problem. Any intentional attempts at trying to obtain the testing set will get you disqualified.

8. Precautions

- Which score is effective: Contestants can select up to 2 submission results for scoring (√ - selected, □ - not selected). The score before unification for this task will be determined by the higher score on the Leaderboard B among the two selected submissions. Other cases of score calculation: please refer to **Appendix Platform Mechanisms and Restrictions for Individual Contest & GAITE**.



- How to deal with ambiguity: Once there is a conflict between the task description and the training set, the validation set and test set , the dataset will be respected first, and the dataset will not be changed during the competition.
- Contestants can only access Leaderboard A during the contest and cannot access Leaderboard B. The final score will be calculated only based on the score in Leaderboard B.
- The highest score by the Scientific Committee for this task is 0.54, this score is used for score unification.

- The baseline score by the Scientific Committee for this task is 0.20, this score is used for score unification.

<< FINAL PAGE >>

<< FINAL PAGE >>